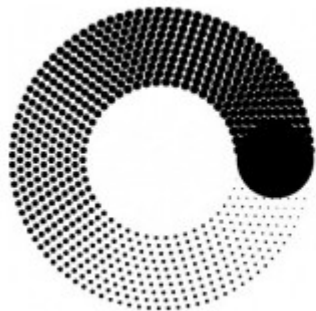




**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 1.
Техническое задание**

**Москва
2024**

Теоретические основы

Техническое задание (ТЗ) — исходный документ на проектирование технического объекта (изделия).

ТЗ устанавливает основное назначение разрабатываемого объекта, его технические характеристики, показатели качества и технико-экономические требования, предписание по выполнению необходимых стадий создания документации (конструкторской, технологической, программной и т. д.) и её состав, а также специальные требования.

Цели и задачи ТЗ

Заказчик:

- Четко сформулировать свои пожелания к будущему продукту
- Выполнить попутную проверку готового продукта
- Требовать от исполнителя соответствия продукта всем условиям, согласно ТЗ

Исполнитель:

- Получить список требований для создания продукта
- Составить план работ по созданию продукта
- Отказаться от выполнения работ, не указанных в ТЗ
- Избежать ошибок, связанных с изменением требований

Выбор шаблона для ТЗ

При выборе шаблона для ТЗ можно руководствоваться следующими документами:

- [ГОСТ 34.602-89](https://online.mospolytech.ru/mod/resource/view.php?id=485940) (https://online.mospolytech.ru/mod/resource/view.php?id=485940)
- [IEEE Std 830](https://online.mospolytech.ru/mod/resource/view.php?id=485941) (https://online.mospolytech.ru/mod/resource/view.php?id=485941)
- Корпоративный шаблон Заказчика

Возможная структура ТЗ

1. Общие положения
2. Назначение и цели создания продукта
3. Характеристика объекта автоматизации
4. Требования к продукту
5. Состав и содержание работ по созданию продукта
6. Порядок контроля и приемки системы (дедлайны)
7. Требования к документированию
8. Источники разработки

Общие положения ТЗ

Первым разделом ТЗ является раздел «Общие положения». В данный раздел могут входить следующие положения:

1. Полное наименование продукта и его условное обозначение
2. Шифр темы или номер договора
3. Наименование заказчика и разработчика продукта
4. Перечень документов, на основании которых создается система
5. Плановые сроки начала и окончания работы по созданию системы
6. Сведения об источниках и порядке финансирования работ
7. Порядок оформления и предъявления заказчику результатов работ
8. Уточнение и дополнение ЧТЗ (частное ТЗ) на систему

Назначение и цели создания системы

В данный раздел могут входить следующие положения:

1. Назначение продукта
2. Цели создания продукта

Характеристика объекта автоматизации

В данный раздел могут входить следующие положения:

1. Описание объекта автоматизации
2. Участники процесса

Требования к продукту

В данный раздел могут входить следующие положения:

1. Требования к продукту в целом (например, структура, функционирование, надежность, безопасность, etc)
2. Требования к видам обеспечения (например, информационное, лингвистическое, etc)
3. Требование к ПО
 - 3.1. Общие требования
 - 3.2. Специфические (например, порядок ведения журнала событий, требования к пользовательским разделам, etc)

Состав и содержание работ по созданию продукта

Данный раздел может содержать информацию о составе, содержании и сроках выполнения этапов работ по созданию продукта по настоящему ТЗ.

Порядок контроля и приемки системы

Данный раздел может содержать в себе следующую информацию:

1. Виды, состав, объем и методы испытаний
2. Требования к проведению предварительных испытаний
3. Требования к проведению опытной эксплуатации
4. Требования к проведению приемочных испытаний
5. Требования к проведению комплексных приемочных испытаний
6. Требования к передаче дистрибутивов
7. Общие требования к приемке работ по этапам

Требования по документированию

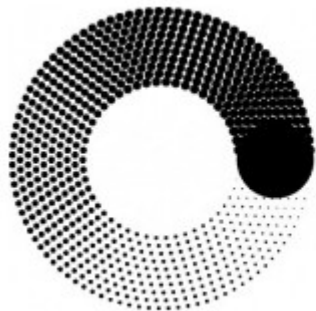
Данный раздел может содержать в себе положения о порядке передачи документации (форма, локализация, носитель) и перечень подлежащих разработке документов.

Источники разработки

В данном разделе необходимо перечислить все использованные при составлении ТЗ источники, в том числе ГОСТы, РД и дополнительные нормативные акты (при наличии).

Задания

1. Разбиться на бригады по 2-3 человека
2. Выбрать вариант проекта для реализации из предложенных вариантов
3. Подготовить Техническое задание для выбранного проекта по выбранному шаблону



**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 2.
Виртуализация**

**Москва
2024**

Теоретические основы

Виртуализация — это предоставление набора вычислительных ресурсов или их логического объединения, абстрагированное от аппаратной реализации, и обеспечивающее при этом логическую изоляцию друг от друга вычислительных процессов, выполняемых на одном физическом ресурсе.

К преимуществам виртуализации при разработке программного обеспечения можно отнести:

- централизованное управление парком виртуальных машин сокращает
- временные и трудозатраты по настройке инстансов;
- меньшие затраты на оборудование - несколько виртуальных машин можно
- развернуть на одной физической машине;
- облегченная миграция;
- возможность изоляции ОС виртуальной машины и ОС хостовой машины;
- возможность создания тестового окружения с различными ОС;
- возможность быстрого восстановления.

Виртуализация бывает следующих видов:

Аппаратная виртуализация — позволяет создавать независимые и изолированные друг от друга виртуальные компьютеры с помощью программной имитации ресурсов (процессора, памяти, сети, диска и др.) физического сервера.

Виртуализация рабочих столов — позволяет отделить логический рабочий стол (набор пользовательских программ, работающий под ОС) от физической инфраструктуры (например, персональных компьютеров). Одной из наиболее распространенных форм виртуализации рабочих столов является VDI (Virtual Desktop Infrastructure).

Виртуализация на уровне ОС (контейнеризация) — позволяет запускать программное обеспечение в изолированных на уровне операционной системы пространствах. Наиболее распространенной формой виртуализации на уровне ОС являются контейнеры (например, Docker).

Основными программными средствами, позволяющими создать виртуальную среду в Windows, являются:

VMWare (<https://www.vmware.com/>)

VirtualBox (<https://www.oracle.com/virtualization/technologies/vm/downloads/virtualbox-downloads.html>)

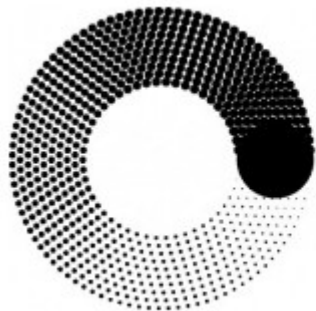
Задания

1. Установить гипервизор на выбор студента
2. Создать несколько виртуальных машин (от 3х VM) с Unix-подобными ОС для стендов test, stage и prod соответственно
3. Объединить созданные виртуальные машину в единую локальную сеть (ping должен работать по принципу «от всех ко всем»)
4. Разместить на тестовом стенде файлы из удаленного репозитория проекта (при помощи утилиты Git)
5. Развернуть на стендах средства разработки, согласно выбранному для проекта стеку. Если выбраны скриптовые ЯП, то запустить прототип, в ином случае - собрать приложение.
6. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).

Вопросы для изучения

На данные вопросы необходимо ответить при защите лабораторной работы.

1. Что общего и чем отличаются различные типы сетей для виртуальных машин (virtual network, bridge, nat)?
2. Каким образом осуществляется обработка системных вызовов Unix в виртуальных машинах при их развертке в ОС Windows?
3. Как размещаются на жестком диске хостовой машины виртуальные жесткие диски виртуальной машины, если их файловые системы различны?



**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 3.
Контейнеризация**

**Москва
2024**

Теоретические основы

Docker

Docker — это программная платформа с открытым исходным кодом, которая помогает упростить процесс создания, управления, запуска и распространения приложений. С помощью Docker можно упаковать свое приложение вместе со всеми его зависимостями в контейнер. Контейнеры позволяют легко и единообразно развертывать ваши приложения.

Компании уже давно используют контейнерные технологии для устранения слабых мест виртуальных машин. Контейнеры в некотором смысле являются подобием облегченных версий виртуальных машин. Важное различие между контейнерами и виртуальными машинами заключается в том, что контейнерам не нужна собственная операционная система. Все контейнеры на узле совместно используют операционную систему хоста, что высвобождает много системных ресурсов.

То, что сейчас известно как современные контейнеры, начиналось как контейнеры Linux (LXC). Многие компании, в том числе Google, помогли внедрить технологии, связанные с контейнерами, в ядро Linux. Без этих вкладов у нас не было бы той богатой экосистемы контейнеров, которую мы имеем сегодня.

Контейнеризация была не очень доступна, пока не появился Docker. Контейнеры Docker создают абстракцию на уровне приложения. Контейнеры упаковывают приложение и его зависимости со всем необходимым для запуска, включая:

- операционную систему
- код приложения
- окружение
- системные инструменты
- системные библиотеки и другое

Образы

Образ — это шаблон с инструкциями по созданию контейнера Docker. Часто образ основано на другом образе с некоторой дополнительной настройкой. Например, вы можете создать образ на основе образа ubuntu, но установить веб-сервер Apache и ваше приложение, а также детали конфигурации, необходимые для запуска вашего приложения.

Вы можете создавать свои собственные образы или использовать только те, которые созданы другими и опубликованы в реестре. Чтобы создать собственный образ, вы создаете Dockerfile с простым синтаксисом для определения шагов, необходимых для создания образа и его запуска. Каждая инструкция в Dockerfile создает слой в образе. Когда вы меняете Dockerfile и перестраиваете образ, перестраиваются только те слои, которые изменились.

Это часть того, что делает образы такими легкими, маленькими и быстрыми по сравнению с другими технологиями виртуализации.

Контейнеры

Контейнер — это исполняемый экземпляр образа. Вы можете создавать, запускать, останавливать, перемещать или удалять контейнеры с помощью Docker API или CLI. Вы можете подключить контейнер к одной или нескольким сетям, подключить к нему хранилище или даже создать новый образ на основе его текущего состояния.

По умолчанию контейнер относительно хорошо изолирован от других контейнеров и своего хост-компьютера. Вы можете контролировать, насколько изолированы сеть контейнера, хранилище или другие базовые подсистемы от других контейнеров или от хост-компьютера.

Контейнер определяется образом из которого он был запущен, а также любыми параметрами конфигурации, которые вы предоставляете ему при создании или запуске. При удалении контейнера любые изменения его состояния, не сохраненные в постоянном хранилище, исчезают.

Основные термины docker

Пространство имен: пространства имен создаются при запуске контейнера. Они обеспечивают уровень изоляции, поскольку каждый элемент контейнера работает в своем пространстве имен.

Cgroups: группы управления позволяют распределять ресурсы между процессами, работающими в системе.

Docker Engine: это основной продукт Docker, который включает в себя демон и интерфейс командной строки. У него есть API для взаимодействия с демоном Docker.

Dockerfile: текстовый документ, содержащий инструкции по созданию образов Docker.

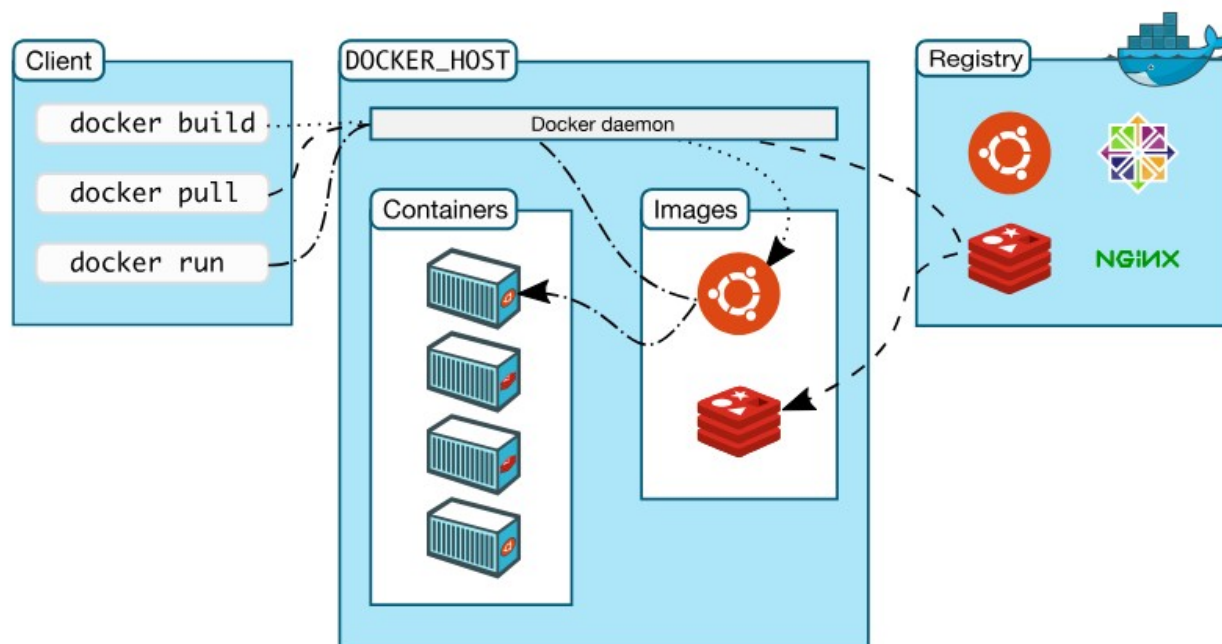
Docker registry (реестр): Реестр Docker позволяет хранить и распространять именованные образы Docker. Реестры организованы в репозитории и содержат все версии различных образов.

Docker Hub: сервис, публичный реестр Docker, который позволяет находить контейнеры и делиться ими.

Docker tag (теги). Теги Docker похожи на метки версии, которые вы можете присвоить любой завершенной сборке (образу).

Объединенная файловая система (AUFS): объединенная файловая система объединяет несколько каталогов на одном хосте и представляет их как один каталог.

Архитектура docker



Основные команды docker

docker pull — это команда, позволяющая загрузить определенный образ или набор образов.

docker run — команда запуска позволяет создать контейнер из указанного образа и запустить этот контейнер с помощью заданной команды.

docker build — это команда, которая используется для создания образа из Dockerfile.

docker stop — это команда, которая используется для остановки контейнера.

docker ps — команда просмотра запущенных контейнеров.

docker images — команда просмотра сохраненных образов

Задания (Часть 1)

1. Установить Docker на всех развернутых в ходе ЛР2 виртуальных машинах
2. На усмотрение студента выбрать Docker-образ, который будет являться основой для разрабатываемого приложения
3. Получить образ на TEST-виртуальную машину
4. Добавить в корень проекта файл Dockerfile, внутри которого происходит сборка и запуск проекта из исходных файлов
5. Собрать приложение внутри Docker, запустить контейнер из полученного образа

Docker Compose

Создать контейнер из образа довольно просто, выполнив одну команду. Docker-compose еще больше автоматизирует это.

Docker-compose вращается вокруг файла конфигурации с именем `docker-compose.yml`. В нем мы определяем все наши сервисы. Сервис это часть вашего приложения; база данных или API, например. Все сервисы полагаются на образы, с помощью которых мы создаем контейнеры. Запуск контейнера может иметь множество вариантов; то, как эти параметры настроены, будет храниться в файле `yml`.

Примером этих параметров запуска является сопоставление портов при запуске контейнера. При обычном использовании `docker` нам пришлось бы вызвать

```
docker run -p 5000:5000 python-docker
```

в нашем терминале для запуска условного сервиса на 5000 порту. Docker-compose позволяет нам определить эти параметры в файле. Еще одним преимуществом наличия всех наших сервисов со всеми соответствующими параметрами сборки в одном файле является то, что мы можем создавать и запускать все наши сервисы одновременно, вызывая

```
docker-compose up
```

Запуск наших контейнеров можно сделать более гибким с помощью переменных среды. Мы можем предоставить `docker-compose` файл, содержащий некоторые переменные среды. Таким образом, мы можем безопасно предоставлять пароли без их жесткого кодирования в файле конфигурации.

Пример: мы создадим файл с именем `.env`, в котором зададим

```
DBPASSWORD=secretpass
```

Затем мы можем использовать `DBPASSWORD` в качестве переменной в файле `docker-compose.yml`.

Для этого мы указываем наш файл `env` при вызове `docker-compose`:

```
docker-compose --env-file .env up
```

Это убережет наши пароли от попадания в репозитории через запись в `docker-compose.yml`. Кроме того, это добавляет больше гибкости и делает проект более аккуратным, поскольку наши пароли больше не жестко запрограммированы.

Общая контейнерная сеть

Когда мы запускаем наши контейнеры с помощью docker-compose, он определяет сеть, которую совместно используют все наши сервисы. Это означает, что все сервисы могут обмениваться данными внутри.

Проиллюстрируем это на примере. Представьте, что мы разместили наше приложение на сайте polytech.mos. Когда нашему API необходимо взаимодействовать с нашей базой данных, ему не нужно подключаться через polytech.mos:5432, а вместо этого можно просто вызвать базу данных изнутри сети docker-compose. Это положительно влияет на безопасность, потому что это означает, что только наши службы имеют доступ к нашей базе данных, клиенты не могут получить доступ к службе из-за пределов приложения.

Переносимость и контроль версий

Поскольку у нас есть вся наша конфигурация в одном файле, мы можем легко поделиться этим файлом через систему контроля версий, такую как Git. Пользователи могут просто извлечь файл docker-compose.yml и исходный код, и все контейнеры будут запущены! Дополнительным преимуществом является то, что мы храним полную историю всех изменений, которые мы вносим в нашу конфигурацию, чтобы мы могли восстановить предыдущую версию, когда захотим. Это также упрощает настройку конвейера CI/CD для этого приложения.

Гибкость

Так как все наши сервисы полностью изолированы друг от друга, мы можем легко добавить новый сервис. Возможно, в будущем нашему приложению понадобится кэширование, просто добавим контейнер Redis! Другие службы, такие как наш API, могут легко подключаться к новой службе Redis по внутренней сети.

Пример файла docker-compose.yml

```
version: '3'

services:

  mospoly_database: #Имя сервиса
    container_name: mospoly_database #Имя с которым запустится контейнер
    hostname: mospoly_database #Имя "хоста" которым будет считать себя контейнер
    image: postgres #Образ из которого запускается контейнер
    volumes:
#Подключаемые директории
      - ./volume:/var/lib/postgresql #Директории вида хост_директория:
директория_контейнера

    environment: #переменные окружения
      - POSTGRES_DB=mospolydb, mospolydb_dev
      - POSTGRES_USER=mospolyuser
      - POSTGRES_PASSWORD=averyStrongPassword
    ports: # Сопоставление портов хост-машины и контейнера
      - 54321:5432
    restart: unless-stopped #условие для рестарта в случае остановки контейнера

  mospoly_api:
    container_name: mospoly_api
    hostname: mospoly_api
    build: # Параметры для сборки приложения
      context: ./mospoly_api # Каталог, используемый для сборки
      dockerfile: Dockerfile # Файл, используемый для сборки образа приложения
    ports:
      - 54322:5000
    volumes:
      - ./mospoly_api/src:/usr/src/app/
      - /usr/src/app/node_modules
    restart: unless-stopped
    environment:
      NODE_ENV: ${MOSPOLY_ENVIRONMENT}
    depends_on: # Указание на зависимость сервиса от другого сервиса
      - mospoly_database

  mospoly_webserver:
    container_name: mospoly_webserver
    hostname: mospoly_webserver
    build:
      context: ./mospoly_webserver
      dockerfile: Dockerfile
    ports:
      - 80:80
      - 443:443
    volumes:
      - ./mospoly_webserver/src/test:/usr/share/nginx/html
    restart: always
    depends_on:
      - mospoly_database
```

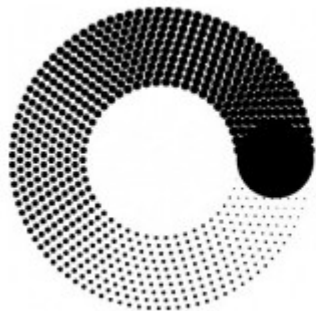
Задания (Часть 2)

1. Установить на все ВМ утилиту docker-compose
2. На выбор студента подобрать подходящую для приложения СУБД и получить с Dockerhub ее официальный (либо сторонний) образ. В случае, если приложение не использует БД, выбрать любой сторонний внешний сервис, используемый приложением (Redis, Minio и др.)
3. В корне проекта создать файл docker-compose.yml, в котором описать параметры для сборки и запуска разрабатываемого приложения а также запуска стороннего сервиса, выбранного студентом. В конфигурации указать подключаемые директории (volume), в которых будут храниться файлы запущенных приложений (логи, файлы БД и другие).
4. Запустить приложение и продемонстрировать его работу со сторонним сервисом.
5. Внести изменения, требующие записи информации в файлы приложения (добавить записи в базу данных через приложение, создать файл и др.)
6. Остановить и запустить приложение еще раз, продемонстрировать, что изменения, внесенные в ходе работы, были сохранены.
7. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).

Вопросы для изучения

На данные вопросы необходимо ответить при защите лабораторной работы.

1. Каким образом docker обеспечивает изоляцию контейнеров от хостовой ОС и друг от друга?
2. Какие параметры требуется передать контейнеру docker при запуске, чтобы его сетевой адаптер был тождественен сетевому адаптеру хостовой ОС и почему?
3. Какую последовательность команд требуется ввести, чтобы при помощи docker-compose сначала собрать приложение, затем запустить его в фоновом режиме и затем подключиться к его выводу stdout и stderr?



**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 4.
Непрерывная интеграция**

**Москва
2024**

Теоретические основы

Непрерывная интеграция (Continuous Integration, CI) — это практика автоматизации интеграции изменений кода от нескольких участников в один программный проект. Это основная передовая практика непрерывной разработки, позволяющая разработчикам часто объединять изменения кода в центральный репозиторий, где затем выполняются сборки и тесты. Автоматизированные инструменты используются для проверки правильности нового кода перед интеграцией.

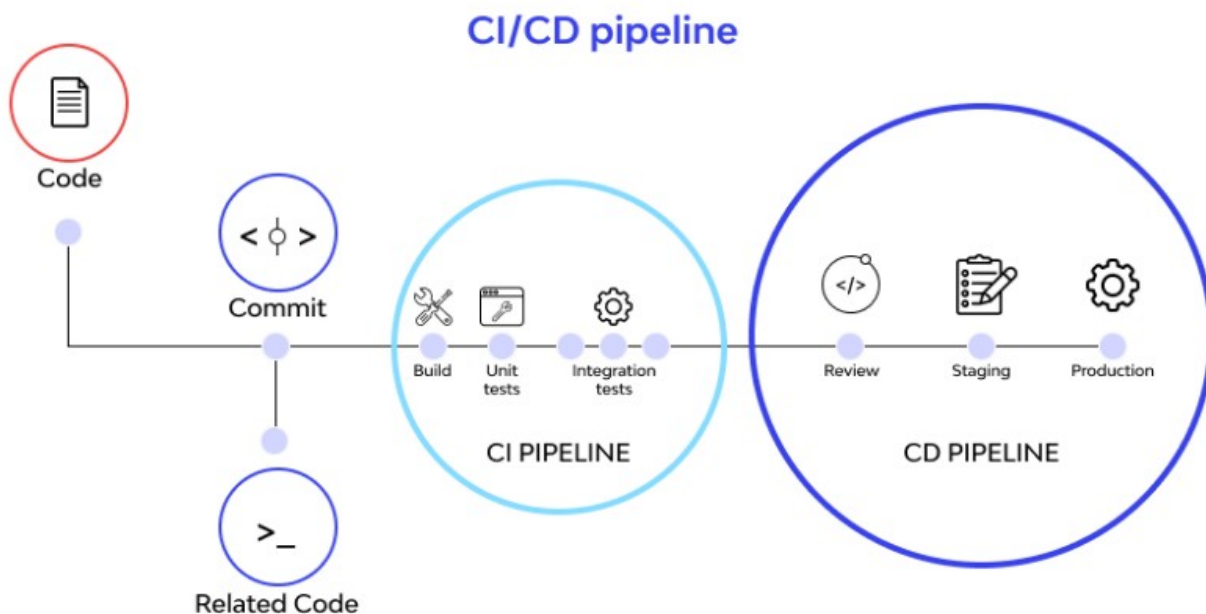
Чтобы понять важность CI, полезно сначала обсудить некоторые болевые точки, которые часто возникают из-за отсутствия CI. Без CI разработчики должны вручную координировать и сообщать, когда они вносят свой код в конечный продукт. Эта координация распространяется не только на команды разработчиков, но и на остальную часть организации. Продуктовые группы должны координировать, когда последовательно запускать функции и исправления, а также какие члены команды будут нести за это ответственность.

Накладные расходы на связь в среде без CI могут превратиться в сложную и запутанную рутинную синхронизацию, которая увеличивает ненужные бюрократические затраты на проекты. Это приводит к более медленным выпускам кода с более высоким уровнем отказов, поскольку требует от разработчиков деликатности и внимательности к интеграции. Эти риски растут в геометрической прогрессии по мере увеличения размеров команды разработчиков и кодовой базы.

Без надежного конвейера непрерывной интеграции может образоваться разрыв между инженерной командой и остальной частью организации. Связь между продуктом и инженерами может быть сложной. Инженерия становится черным ящиком, в который остальная часть команды вводит требования и функции и, возможно, получает ожидаемые результаты. Инженерам будет сложнее оценивать время доставки по запросам, потому что время для интеграции новых изменений становится неизвестным риском.

Непрерывная интеграция помогает увеличить численность персонала и производительность инженерных групп. Внедрение CI в вышеупомянутый сценарий позволяет разработчикам программного обеспечения параллельно работать над функциями независимо друг от друга. Когда они будут готовы объединить эти функции в конечный продукт, они смогут сделать это независимо и быстро. CI — ценная и хорошо зарекомендовавшая себя практика в современных высокопроизводительных организациях по разработке программного обеспечения.

Схематически процесс непрерывной интеграции (и непрерывной доставки (Continuous delivery, CD), которая будет рассмотрена позже), можно отобразить следующим образом:



Задача инструментов CI/CD заключается в координации действий между целым рядом систем, включая системы контроля версий (например, Git, Mercurial или Perforce), инструменты сборки (например, Ant, Maven или Gradle), автоматизированные среды тестирования (например, NUnit или JUnit), менеджеры пакетов (например, NuGet), средства отслеживания проблем (например, Jira, YouTrack или Github) и контейнерные платформы (например, Docker).

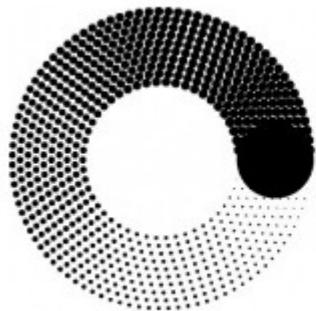
Когда дело доходит до построения конвейера CI/CD, универсального подхода не существует, поэтому очень важно иметь возможность интеграции с различными инструментами, будь то через встроенную поддержку, сторонние плагины или API, которые позволяют расширить функциональность в соответствии с вашими потребностями. Наличие такой гибкости упрощает расширение вашей платформы CI/CD для других команд в вашей организации в будущем.

Задания

1. При помощи команды `docker push` собранный в ходе ЛР3 образ разрабатываемого приложения загрузить в DockerHub
2. Загрузить с Docker Hub образ `jetbrains/teamcity-server` и развернуть его согласно инструкции на странице образа
3. Открыть веб-интерфейс развернутого приложения, завести пользователя-администратора
4. В панели администрирования завести по одному пользователю для каждого члена команды
5. Завести репозиторий в TeamCity, привязав репозиторий проекта
6. По выбору студента - либо из предложенных вариантов сборки, либо из реализованных студентом создать конфигурацию (последовательность шагов)

для сборки разрабатываемого приложения (для всех веток) в Docker образ и отправки его в Docker Hub (для ветки «в разработке» (dev) и ветки «в эксплуатации» (prod))

7. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).



**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 5.
Непрерывная доставка**

**Москва
2024**

Теоретические основы

Непрерывная доставка (Continuous delivery, CD) является расширением непрерывной интеграции, поскольку она автоматически развертывает все изменения кода в тестовой и/или производственной среде после этапа сборки.

Это означает, что помимо автоматического тестирования у вас есть автоматизированный процесс выпуска, и вы можете развернуть свое приложение в любое время, нажав кнопку.

Теоретически при непрерывной доставке возможно выпускать обновления продукта ежедневно, еженедельно, раз в две недели или в зависимости от бизнес-требований организации. Однако, если необходимо действительно воспользоваться преимуществами непрерывной доставки, развертывание в рабочей среде следует выполнять как можно раньше, чтобы убедиться, что изменения между обновлениями минимальны и, в случае проблем, их можно легко откатить.

CD — это комплексная методология для всей организации, которая включает неинженерные команды, такие как дизайнеры, продуктологи и маркетологи.

Автоматизация является ключевой ценностью CD. Человеческое время стоит дорого, и его следует тратить на творческие задачи, а не на утомительное выполнение задач по инструкции. Ручной процесс не является по-настоящему воспроизводимым и надежным, пока он не закреплен за кодом и не может выполняться автоматически по запросу. Автоматизированные задачи могут быть объединены вместе для создания дополнительных уровней автоматизации.

Максимально необходимо автоматизировать: тесты, релизы, изменения конфигурации и многое другое

Задания

1. Добавить к созданным в ходе ЛР4 шагам шаги, осуществляющие развертывание приложения на stage-сервере из образа, загруженного в Docker Hub вместе с базой данных (либо иным сторонним сервисом, добавленным в docker-compose.yaml в ходе ЛР3). Состав шагов и их наполнение выбираются студентом в зависимости от выбранного типа приложения.

2. Развернуть на test-стенде образ JetBrains/teamcity-agent, настроить его для работы с уже запущенным сервером

3. Внести в репозиторий какое-либо изменение, касающееся функционала разрабатываемого приложения, в соответствии с требованиями внесения изменений GitFlow, либо другой методологии на выбор студента. В дальнейшем

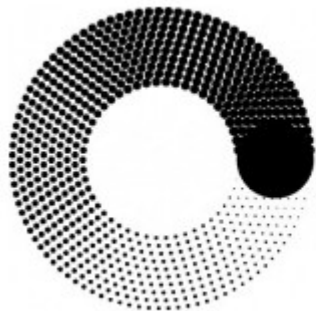
все изменения в код репозитория необходимо вносить согласно выбранной методологии.

4. Зафиксировать успешную сборку приложения из кода, добавленного в feature-ветку в TeamCity

5. Создать пул-реквест созданной ветки в ветку dev с требованием согласования со стороны другого члена команды

6. После согласования и создания коммита, зафиксировать сборку приложения из кода в ветке dev и развертывание приложения из созданного образа на stage-сервере

7. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).



**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 6.
Статический анализ кода**

**Москва
2024**

Теоретические основы

Статический анализ кода (Static Code Analysis) — анализ исходного кода программного обеспечения, производимый без реального выполнения исследуемых программ (в отличие от динамического анализа). В большинстве случаев анализ производится над исходным кодом, хотя, иногда анализу подвергается объектный код, например Р-код или код на MSIL. Термин обычно применяют к анализу, производимому специальным программным обеспечением (ПО), тогда как ручной анализ называют «program understanding», «program comprehension» (пониманием или постижением программы).

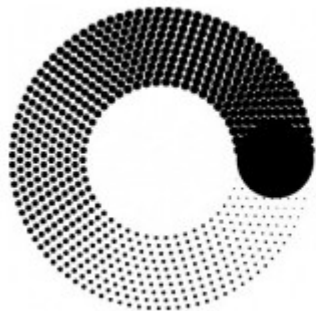
В зависимости от используемого инструмента глубина анализа может варьироваться от определения поведения отдельных операторов до анализа всего имеющегося кода. Способы использования полученной в ходе анализа информации также различны — от выявления мест, возможно содержащих ошибки (утилиты типа Lint), до формальных методов, позволяющих математически доказать какие-либо свойства программы (например, соответствие поведения спецификации).

Типы ошибок, обнаруживаемых статическими анализаторами

- **Неопределённое поведение** — неинициализированные переменные, обращение к NULL-указателям. О простейших случаях сигнализируют и компиляторы.
- **Нарушение алгоритма пользования библиотекой.** Например, во многих языках программирования каждому открытию файла должно соответствовать его закрытие, поэтому если файловая переменная теряется раньше, чем файл закрывается, анализатор может сообщить об ошибке.
- **Сценарии, приводящие к недокументированному поведению.**
- **Переполнение буфера,** чаще всего вызванное использованием небезопасных функций или неопределённым поведением.
- **Типичные сценарии, мешающие кроссплатформенности.**

Задания

1. Провести исследования доступных статических анализаторов для выбранного командой стека разработки (большой, но не всеобъемлющий список ([https://owasp.org/www-community/Source Code Analysis Tools](https://owasp.org/www-community/Source_Code_Analysis_Tools)))
2. Выбрать минимум два из следующих видов статических анализаторов:
 - а. анализатор качества кода, например, Checkstyle
 - б. анализатор безопасности кода, например, Gosec
 - с. средство поиска учетных данных/секретов, например, Trufflehog.
3. Добавить в построенном пайплайне TeamCity шаги, на которых код проверяется статическими анализаторами и, в случае обнаружения проблем максимальной критичности для выбранного анализатора (для анализатора безопасности — CRITICAL, для анализатора качества кода — BLOCKER, для средства поиска секретов — любой обнаруженный пароль), билд завершается с ошибкой.
4. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).



**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 7.
Автоматизированное тестирование**

**Москва
2024**

Теоретические основы

Автоматизированное тестирование лежит в основе любого конвейера CI/CD. Это связано с тем, что вся концепция CI/CD вращается вокруг принципа «быстро строить, быстро тестировать, быстро ошибаться».

Тесты должны выполняться как можно быстрее, чтобы результаты доходили до разработчика как можно раньше. Это позволяет своевременно обнаруживать ошибки. В результате продукт, выходящий на стадию релиза, будет более качественным и лишенным очевидных ошибок. Все это будет возможно только с автоматизацией тестирования ПО.

Автоматизация тестирования дает следующие преимущества:

1. Сокращает ручные усилия, когда один и тот же тест необходимо запускать повторно. Это освобождает больше времени для выполнения тщательных ручных тестов.
2. Дает немедленную обратную связь.
3. Тесты могут быть более точными и охватывать больше тестовых случаев.
4. Помогает генерировать и сравнивать результаты нескольких тестов, обеспечивая согласованность продукта.
5. Становится неотъемлемой частью конвейера CI/CD, соблюдая принцип «быстро строить, быстро ошибаться».

Очень важно иметь несколько наборов автоматизированных тестов, созданных для разных целей. Запуск одного набора тестов, охватывающего все сценарии, только замедлит процесс.

Ниже приведены некоторые типы тестирования и их место в конвейерах CI/CD:

Юнит тесты

Многие команды используют подход Test Driven Development (TDD). Юнит тесты пишутся разработчиками и запускаются сразу после этапа сборки. **Модули тестируются отдельно друг от друга.**

Интеграционные тесты

После сборки и развертывания каждого коммита в среде разработки эти тесты запускаются, чтобы проверить, хорошо ли взаимодействуют новые добавленные модули/изменения. **Тестируется совместная работа модулей**

Регрессионные тесты

Регрессионные тесты запускаются один раз в некоторое время (например, после окончания рабочего дня), чтобы гарантировать, что новые добавленные изменения не повлияют на существующий код. Это помогает убедиться, что дневная работа прошла хорошо, и дает обратную связь, если необходимы какие-либо изменения. **Все, что было написано ранее, тестируется, даже если оно было протестировано недавно.**

Нагрузочные тесты

Перед выпуском кода в производственную среду выполняется набор тестов для оценки скорости отклика (производительности) и стабильности системы. Эти тесты выполняются в среде предпроизводственной среде после развертывания кода после внесения существенных изменений. **Тестирование не фактического получения верного результата теста, но и скорости его получения.**

Это тесты, которые в идеале должен иметь каждый конвейер CI/CD, и, по сути, они должны быть автоматизированы. По мере того, как продукт растет и предлагает больше функций и обновлений, набор тестов также увеличиваются в геометрической прогрессии. Запустить все вышеперечисленные тесты вручную было бы невозможно, и автоматизация — единственный возможный способ выполнить их быстро и точно.

В составе CI/CD пайплайна место автоматизированных тестов находится между сборкой (на которой получаем работающий экземпляр ПО - контейнер или бинарный файл) и доставкой приложения на новый стенд, таким образом, при наличии проблем, обнаруженных автоматическими тестами, пайплайн обрывается и некачественный код не попадает ни на тестовый стенд, ни на производственный стенд.



Пример юнит-теста на языке Go:

```
package arraycounter //указываем тестируемый пакет

import "testing" //импортируем модуль для тестов - testing

func TestCountElementsInArray(t *testing.T){ /* Создаем функцию
                                                с именем Test+ТестируемаяФункция */

    got := CountElementsInArray([]int{1,2,2,3,4,6,6,6,1}, 6) /* Вызываем функцию
                                                                со статическими параметрами */

    want := 3 //Задаем эталонный результат функции

    if got != want { //сравниваем эталонный результат и реальный

        t.Errorf("got %q, wanted %q", got, want) /* если получившийся результат
                                                    не совпал с эталоном, вызываем ошибку*/
    }
}
```

Пример интеграционного теста на языке Go:

```
package arraycounter

import "testing"

func TestReplaceElementsInArray(t *testing.T){ /* Создаем функцию для тестирования
                                                нескольких функций внутри нее */

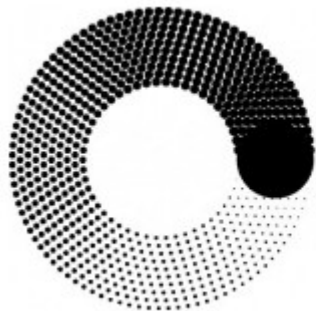
    marks, err := database.GetListOfMarksByStudentId(1) /* Вызываем первую
                                                            тестируемую функцию */

    if err!= nil{ //Проверяем что функция выполнялась без ошибок
        t.Errorf("unable to get marks with error %s", err.Error())
    }
    got := CountElementsInArray(marks, 2) /* Проверяем что результат одной функции
                                                успешно обрабатывается другой функцией */
    want := 3

    if got != want {
        t.Errorf("got %q, wanted %q", got, want)
    }
}
```

Задания

1. Выбрать библиотеку, либо фреймворк для тестирования ПО на выбранном для разработки стеке
2. Разработать набор юнит- и интеграционных тестов для разработанной части ПО, таким образом, чтобы тестирование покрывало от 40% функционала, описанного в техническом задании. В отчете описать каждый отдельный тест (опционально — с примером кода), в частности, описать, какой модуль (модули) тестируется, какой ожидаемый результат, какие данные используются для получения искомого результата.
3. Добавить к разработанным конфигурациям сборки в TeamCity шаги, осуществляющие автоматическое тестирование разработанного кода (независимо от ветки, в которой код находится)
4. Продемонстрировать хотя бы одно неуспешное прохождение теста (с заранее плохим кодом, не тестом) и по одному успешному прохождению шагов тестирования для одной FEATURE/FIX ветки, DEV-ветки и PROD-ветки
5. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).



**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 8.
Использование средств миграции баз данных**

**Москва
2024**

Теоретические основы

Непрерывная интеграция базы данных

Непрерывная интеграция базы данных — это быстрая интеграция изменений схемы и логики базы данных в процесс разработки приложений и предоставление немедленной обратной связи разработчикам по любым возникающим проблемам.

Непрерывная доставка базы данных

Непрерывная доставка базы данных — это процесс, направленный на внесение изменений в развернутую базу данных для реализации новых функций, обновлений и исправлений ошибок как можно быстрее и безопаснее.

Это достигается за счет оптимизации процесса разработки и сохранения кода приложения и кода базы данных в готовом к развертыванию состоянии.

Чтобы быстрее выводить на рынок новое программное обеспечение, компании в последнее десятилетие пытались модернизировать способы создания и доставки программного обеспечения. В своих усилиях по модернизации и улучшению конвейера непрерывной доставки многие инструменты сосредоточились на внедрении CI и CD в процесс изменения кода приложения.

Однако этого не произошло с базами данных. Многие организации по-прежнему полагаются на собственную команду поддержки базы данных, которая вручную проверяет и развертывает изменения БД. Учитывая, что опыт конечного пользователя неполный без полного стека программного обеспечения, который включает в себя как приложение, так и базу данных, растет интерес и потребность в средствах непрерывной интеграции и доставки для базы данных.

Цель инструментов непрерывной интеграции базы данных состоит в том, чтобы привнести в базу данных одни и те же передовые методы интеграции и развертывания, а также обеспечить прохождение кода SQL через конвейер выпуска программного обеспечения, синхронизированного с кодом приложения. Согласовывая развертывания баз данных с развертываниями приложений, команды получают гораздо большую отдачу от своих инвестиций в инструменты и обновления процессов. Это, в свою очередь, помогает командам быстрее и качественнее выводить на рынок новые инновации.

Лучшие практики при интеграции CI/CD для базы данных

Многие передовые методы, применимые к CI приложений и средствам доставки, легко применимы и к CI базы данных. Эти передовые методы включают в себя:

1. Отслеживание изменений кода базы данных

Изменения в коде базы данных должны отслеживаться в той же системе управления исходным кодом или версиями, что и код приложения. Код базы данных не следует обрабатывать отдельно или отслеживать в совершенно другой системе. Отдельная система для кода базы данных приводит к излишним усилиям, отсутствию наглядности, путанице и ошибкам. Код приложения и базы данных также начнет дрейфовать в отдельной системе и смещаться.

2. Автоматическая проверка кода базы данных и обратная связь

Как только код приложения попадает в репозиторий, сразу же запускается серия автоматических тестов. Эти тесты оценивают наличие проблем в коде, требующих доработки. К сожалению, это не относится к коду базы данных в большинстве компаний. Одной из самых больших проблем при ускорении развертывания баз данных является избавление от ручной проверки кода SQL, которую должны выполнять администраторы баз данных.

Эта утомительная ручная работа может и должна быть в значительной степени устранена с помощью интеллектуальной автоматизации. Таким образом разработчики могут получить немедленную обратную связь по коду SQL — так же, как они делают это с кодом приложения — и избежать длительного состояния ожидания, в котором они переходят к другой задаче. Эта неэффективность обратной связи вызывает большие задержки в развертывании баз данных и способствует снижению качества. Это также заставляет разработчиков переключаться между контекстами, чтобы вносить исправления в изменения кода SQL, которые они написали несколько дней или недель назад.

3. Упаковка кода базы данных

Важная часть философии DevOps — «создать один раз, развертывать часто». По сути, все инструменты непрерывной интеграции позволяют встраивать код приложения в неизменяемый пакет для последовательного, воспроизводимого и предсказуемого последующего развертывания. Если что-то пойдет не так, то его ошибки сразу же можно отследить либо в коде приложения, либо в среде при работе с неизменяемым артефактом. Инструменты непрерывной интеграции базы данных должны приносить те же самые улучшения в код базы данных.

4. Обеспечение видимости состояния базы данных

Еще один ключевой принцип DevOps — «усиливать обратную связь». Решения для непрерывной интеграции нацелены на быстрое обеспечение наглядности и обратной связи. Таким образом, решение для непрерывной интеграции базы данных должно иметь доступный интерфейс. Этот интерфейс должен позволить всем заинтересованным сторонам быстро понять состояние каждой базы данных. Кроме того, важно, чтобы решение интегрировалось с системами тикетов, такими как JIRA и TFS, и поддерживало стратегии параллельной разработки, которые обычно используются в корпоративных командах разработчиков.

Задания

1. Создать в репозитории проекта директорию для хранения актуальной схемы базы данных.
2. Провести анализ существующих инструментов статического анализа безопасности SQL-кода, выбрать наиболее удобное, по мнению команды, средство, обосновать выбор.
3. Добавить в CI/CD пайплайны для всех веток шаг, осуществляющий проверку безопасности SQL файлов
4. Внести изменение в код ПО и базу данных, таким образом, изменив актуальную схему БД.
5. Выгрузить при помощи штатных средств СУБД актуальную схему БД и сохранить ее в созданной директории, добавить ее к проекту и зафиксировать изменение коммитом.
6. После успешного тестирования нового функционала на TEST-стенде, вручную внести изменения в схему БД на STAGE-стенде перед развертыванием обновления.
7. Развернуть обновление на STAGE-стенде (при помощи коммита в DEV-ветку).
8. Проверить корректную работу примененной схемы БД и нового обновления приложения.
9. Провести анализ существующих инструментов автоматизации миграции схемы БД в CI/CD пайплайне, выбрать наиболее удобное, по мнению команды, средство, обосновать выбор.
10. Внедрить в CI/CD пайплайн для DEV-ветки шаг(и), осуществляющий автоматическое развертывание новой схемы БД на STAGE-сервере путем предварительной сверки схем БД на TEST-сервере и STAGE-сервере

11. Протестировать работу нового шага в пайплайне путем изменения схемы БД и созданием коммита в DEV-ветке, убедиться, что схемы БД идентичны

12. Создать отдельный пайплайн, идентичный пайплайну для DEV-ветки, но осуществляющий доставку и развертывание на PROD-сервере. Схема БД в данном случае применяется для соответствия STAGE-серверу

13. В созданный пайплайн добавить шаг, осуществляющий резервное копирование информации из БД перед внесением изменений

14. Проверить работу нового пайплайна, продемонстрировать успешное сохранение резервной копии БД

15. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).